Today's Content

1) Max Subarray Sum

2) Zero Queries

3) Max Absolute Difference

Answers → Private chat
Questions → Public chat

Welcome All!

Starting at 9:06 PM

→ Razorpay (1.5 years) (2 months back)

→ Samsung Research

→ 2019 (Thapar University)

→ 4 years of experience.

→ Booking.com in Amsterdam. (5th Feb)

Q) Given N array elements. Find max subarray sum.

↓ continuous part of an array

e.g. arr[7] =	0	1	2	3	4	5	6
	-3	4	6	8	-10	2	7

↳ max subarray sum = 18

arr[5] =	-3	-2	-8	-1	-10

↳ ans = -1

Idea:- Generate all subarrays and get it's sum.

s	e
2	5

s <= e

arr[8] =	0	1	2	3	4	5	6	7
	3	-2	4	6	3	2	7	2
			e	e	e	e	e	e
			s	e	e	e	e	e

```
maxSum = INT_MIN;
for(s=0; s<N; s++){
    // where can my subarray end
    for(e=s; e<N; e++){
        // [s,e] => subarray fixed
        sum=0
        for(i=s; i<=e; i++){
            sum += arr[i];
        }
        maxSum = max(maxSum, sum);
    }
}
```

T.C = O(N^3)

S.C = O(1)

	0	1	2	3	4	5	6	7	8
arr[] =	3	2	-1	6	4	8	3	-2	7

All subarrays starting at index 3

6 +4

10 +8

18 +3

```
sum=0
for(j=3; j<N; j++){
    sum = sum + arr[j];
    print(sum);
}
```

{0,1,2,3,...N-1}

→

```
ans = INT_MIN;
for(i=0; i<N; i++){
    sum=0;
    for(j=i; j<N; j++){
        sum = sum + arr[j];
        ans = max(sum, ans);
    }
}
```

↓

T.C = $O(N^2)$

S.C = $O(1)$

Observations:-

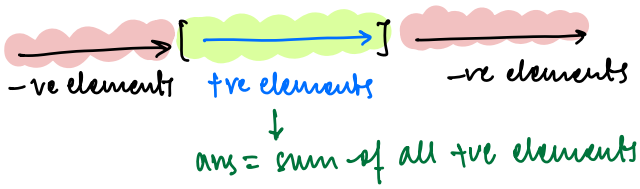
e.g. 1 3 5 6 8 2

if all elements are +ve → max subarray sum = sum of arr[].

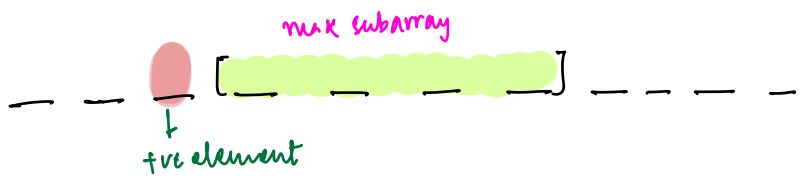
e.g. 2 -2 -8 -9 -11 -10

if all elements are -ve → max subarray sum = max element in arr[]

e.g. 3



e.g. 4



e.g. 5

e.g.1.

	0	1	2	3	4	5	6	7	8	9	10	11
	5	6	7	-3	2	-10	-12	8	12	21	-4	7
sum=0	+5	11	18	15	17	7	~~-5~~ 0	8	20	41	37	44
ans=-∞	+5	11	18	18	18	18	18	18	20	41	41	44 → ans

Note:- if sum < 0 → sum = 0

Eg.2

	10	-20	-12	6	5	-3	8	-2
sum=0	10	~~-10~~ 0	~~-12~~ 0	6	11	8	16	14
ans=-∞	10	10	10	10	11	11	16	16 → ans

e.g.3

	0	1	2	3
	-10	-3	-2	-14
sum=0	~~-10~~ 0	~~-3~~ 0	~~-2~~ 0	-14
ans=-∞	-10	-3	-2	-2 → ans = -2.

Pseudo Code:-

```
ans = INT_MIN;
sum = 0;
for(i=0; i<N; i++){
    sum = sum + arr[i];
    ans = max(ans, sum);
    if(sum < 0){ sum = 0; }
}
return ans;
```

T.C = O(N)

S.C = O(1)

KADANE'S ALGORITHM

Q: Given a arr[N].

Initially arr[] = 0.

Return final array after performing all the queries. {Q queries}

Queries (i, x) = Add x to all the numbers from arr[i] to arr[N-1]

arr[7] = {0}

e.g. A = [0 0 0 0 0 0 0] (i, x) : 3 queries

0 1 2 3 4 5 6

3 3 3 3 3 3 → (1, 3)

4 4 4 → (4, -2)

4 2 2 2 → (3, 1)

A = [0 3 3 4 2 2 2]

arr[7] = {0}

e.g. A = [0 0 0 0 0 0 0] (i, x) : queries

0 1 2 3 4 5 6

6 6 6 6 6 → (2, 6)

-1 -1 5 5 5 5 5 → (0, -1)

7 7 7 7 → (3, 2)

11 11 → (5, 4)

A = [-1 -1 5 7 7 11 11]

idea 1:- For every query [i, x], iterate from arr[i] → arr[N-1] and add x.

↳ T.C = O(Q * N)

S.C = O(1)

idea 2 :-

arr[7] = {0}

e.g. A = [0 0 0 0 0 0 0] (i, x) : 3 queries

A = [0 3 0 1 -2 0 0] → (1, 3)

Apply pf[] = [0 3 3 4 2 2 2] → (4, -2)

→ (3, 1)

arr[7] = {0}

e.g. A = [0 0 0 0 0 0 0] (i, x) : queries

A = [-1 0 2 2 0 0 0] → (2, 6)

Apply pf[] = [-1 -1 1 3 3 3 3] → (0, -1)

→ (5, 2)

→ (2, -4)

TODO → Cross check

Pseudo code:-

```
int[] modify (int arr[], int N, int Q, int mat[Q][2]) {
    Step1:- Iterate on queries and add value x
            to the array elements.

    for(i=0; i<Q; i++){
        i = mat[i][0] ; x = mat[i][1]
        arr[i] += x;
    }

    Step2:- Apply PfSum on your modified arr[]
}
```

For some q^{th} query:-
i = mat[q][0]
x = mat[q][1]

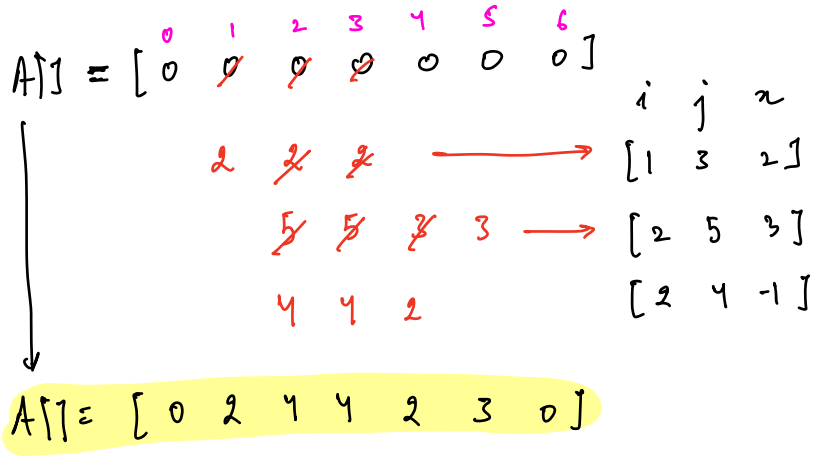
T.C = Q + O(1) + O(N) → O(N+Q)

// directly modifying the i/p array
S.C = prefix in arr[] → S.C: O(1)

// take entire extra array for prefix
prefix using pf[] → S.C = O(N)

arr[N] = 0 { Perform Q queries and return the final array }

Queries → (i, j, x) = Add x to all elements from arr[i] → arr[j]



idea 1:- for every query [i, j, x], add x for all elements [i - j]

T.C = O(Q * N)

S.C = O(1)

idea 2:- Optimisation

0 1 2 ... i i+1 ... j j+1 j+2 ..... N-1

(i, j, x) :- (Add x, i, N-1) x x ... x x x x

Add (-x, j+1, N-1) -x -x ..... -x

(i, j, x) = [Add x from i to N-1], [Add -x from j+1 to N-1]

= Query (i, x)

= arr[i] += x

Query (j+1, -x)

arr[j+1] -= x

Dry Run:-

1) A[] = [0 0 0 0 0 0 0]

arr[i] += x

arr[j+1] -= x

i	j	x
1	3	2
2	5	3
2	4	-1

A[] = [0 2 2 0 -2 1 -3]

↓

Apply p[] = [0 2 4 4 2 3 0]

2) A[] = [0 0 0 0 0 0 0]

i	j	x
2	4	6
5	6	2
1	3	4

arr[7] is out of bounds. No need to subtract.

↓

A[] = [0 4 6 2 -4 -6 0]

p[] = [0 4 10 12 8 2 2]

Pseudo code:-

```
int[] modify (int arr[], int N, int Q, int mat[Q][3]) {
    Step1:- Update all queries.
    for(i=0; i<Q; i++){
        i = mat[i][0]; j = mat[i][1]; x = mat[i][2];
        arr[i] += x;
        if(j+1 < N){
            arr[j+1] -= x;
        }
    }
    Step2:- Apply pf sum to modified array
}
```

→ [i j x]
→
→

T.C = O(Q+N)

S.C = O(1) , O(N)

↓ use i/p arr

↳ use new array.

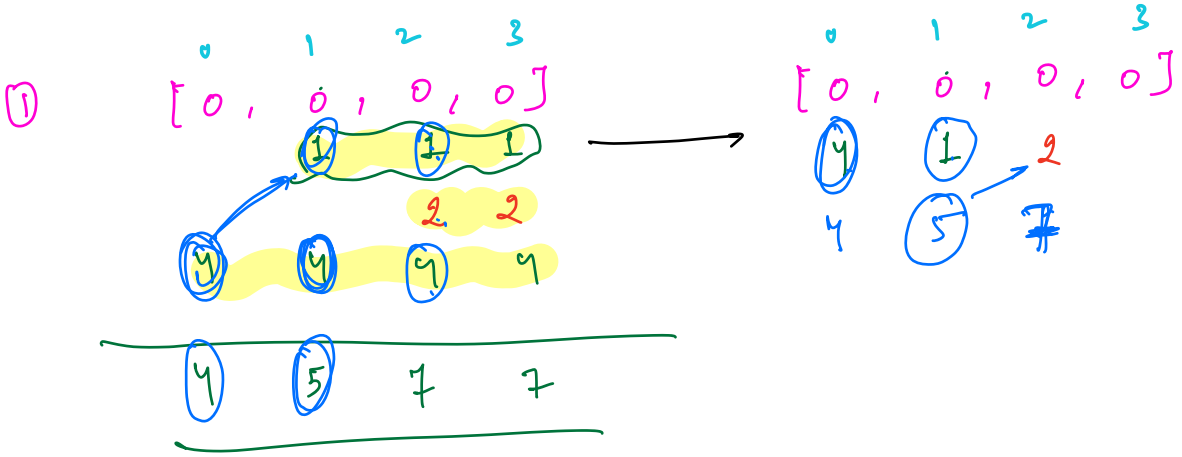
Doubts

```
ans = INT_MIN;   int si=0, ei=0
for(i=0; i<N; i++){
    sum=0;
    for(j=i; j<N; j++){
        sum = sum + arr[j];
        if(sum > ans){
            si=i, ei=j
            ans = sum;
        }
    }
}
return ans;
```

i,j	0	1	2	3	4
	2	3	9	6	4

ans = 2

sum = 0, 2, 5



arr[] = 1 2 3 7 6 10